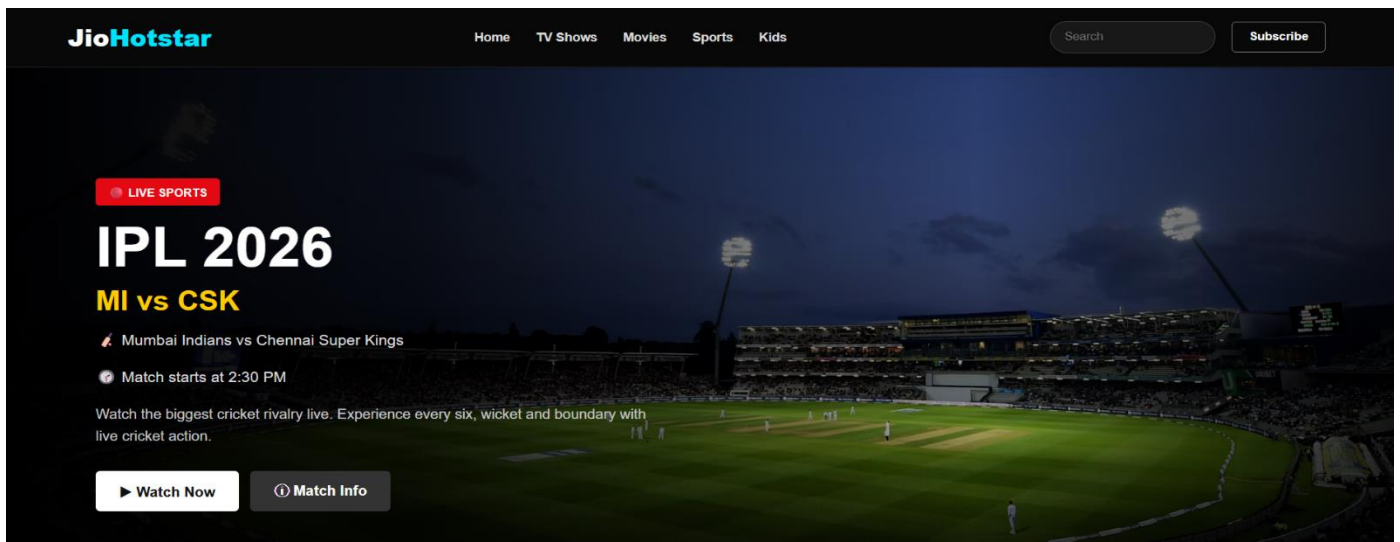




From Code to Container

A production-oriented guide to the JioHotstar-inspired landing page deployment

Technical guide

Create by Mohit

24-8-2026

This document explains what each command does, why it appears in the workflow, when it is appropriate in production, and what should be changed before operating a public service.

Contents

1	Executive summary	1
2	Architecture and deployment flow	1
2.1	End-to-end flow.....	1
3	Visual project showcase	1
4	Phase 1 — Server preparation.....	3
4.1	<code>sudo su</code>	3
4.2	<code>yum update -y</code>	3
4.3	<code>yum install docker -y</code>	4
4.4	<code>which docker</code>	4
4.5	<code>docker --version</code>	4
4.6	<code>systemctl status docker</code>	4
4.7	<code>systemctl start docker</code>	4
4.8	<code>systemctl enable docker</code>	5
5	Phase 2 — Project workspace.....	5
5.1	<code>mkdir jioHotstar</code>	5
5.2	<code>cd jioHotstar</code>	5
6	Phase 3 — Image definition and build	5
6.1	<code>docker pull nginx</code>	5
6.2	<code>vi index.html</code>	6
6.3	<code>touch Dockerfile</code>	6
6.4	Dockerfile instruction: <code>FROM nginx:alpine</code>	6
6.5	Dockerfile instruction: <code>COPY . /usr/share/nginx/html</code>	6
6.6	Dockerfile instruction: <code>EXPOSE 80</code>	6
6.7	Dockerfile instruction: <code>CMD ["nginx", "-g", "daemon off;"]</code>	7
6.8	<code>docker build -t jiohotstar</code>	7
7	Phase 4 — Container launch	7
7.1	<code>docker run -d --name jio-hotstar-server -p 80:80 jiohotstar</code>	7
8	Production readiness checklist.....	8
9	Safer reference implementation.....	8
10	Command summary.....	9
11	Conclusion.....	10
12	References.....	10

1 Executive summary

This project demonstrates a compact path from a static HTML file to a web server running inside a Docker container. The workflow has four distinct concerns: preparing a Linux host, creating a project workspace, defining an immutable image, and launching a container with a host-to-container port mapping.

The commands are useful for learning and for small, controlled deployments. In a production environment, however, the same flow should be strengthened with least-privilege access, pinned package and image versions, a non-root runtime where practical, health checks, externalized configuration, image scanning, logging, TLS termination, backups, and a repeatable deployment pipeline.

Core distinction. A **Dockerfile** is a recipe. An **image** is the packaged result of that recipe. A **container** is a running process created from an image. The command `docker build` creates the image; `docker run` creates and starts the container.

2 Architecture and deployment flow

The implementation serves static files through NGINX. The Dockerfile copies the project directory into NGINX's default document root, exposes port 80 as metadata, and keeps NGINX in the foreground so the container remains alive.

Layer	Artifact or process	Operational responsibility
Host	Linux server, Docker Engine, firewall, users	Patch, secure, monitor, and back up the host
Image	<code>jiohotstar:latest</code> built from the Dockerfile	Build reproducibly, scan, sign, and store in a registry
Container	<code>jio-hotstar-server</code> running NGINX	Publish traffic, observe health, rotate logs, and restart safely
Network	Host port 80 mapped to container port 80	Use TLS, ingress, firewall rules, and a controlled exposure policy

2.1 End-to-end flow

```

Linux host
├── Docker Engine
│   ├── Image: jiohotstar
│   │   └── Container: jio-hotstar-server
│   │       └── NGINX :80 ← host :80

```

3 Visual project showcase

The screenshots below document the visible result of the application that the Docker workflow packages and serves. They provide useful context for the deployment: the project is a static streaming-style interface composed of a hero banner, content discovery sections, sports cards, and subscription plans.

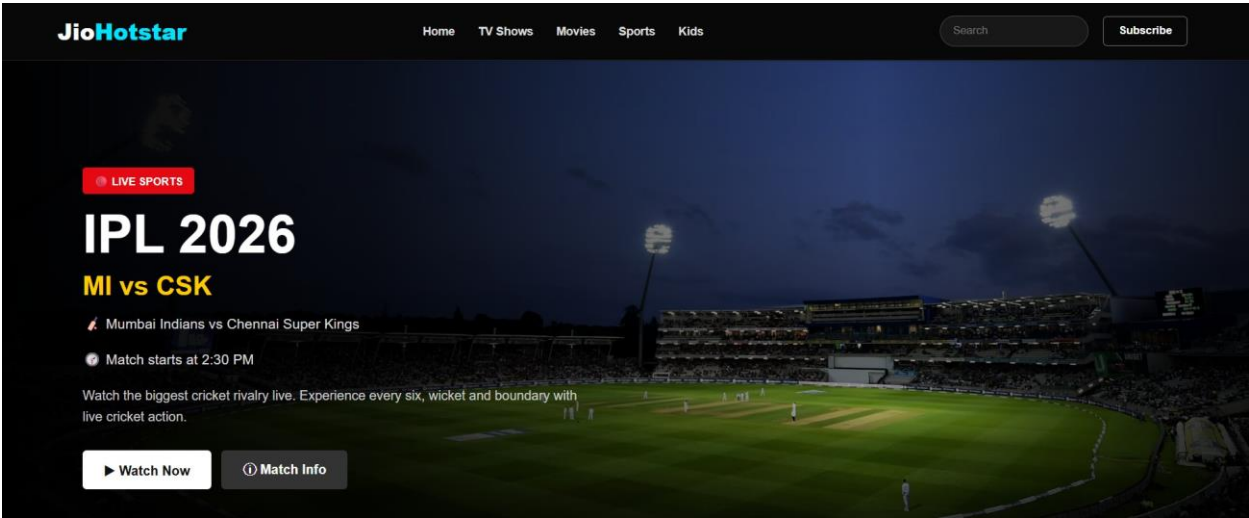


图 1 Landing-page hero section with navigation, live sports messaging, and primary calls to action.

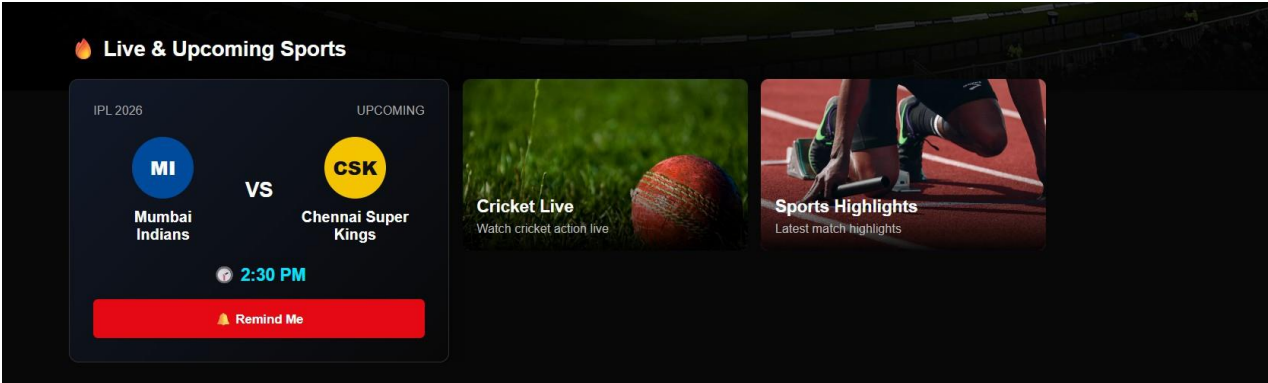


图 2 Live and upcoming sports section demonstrating a match card and supporting content cards.

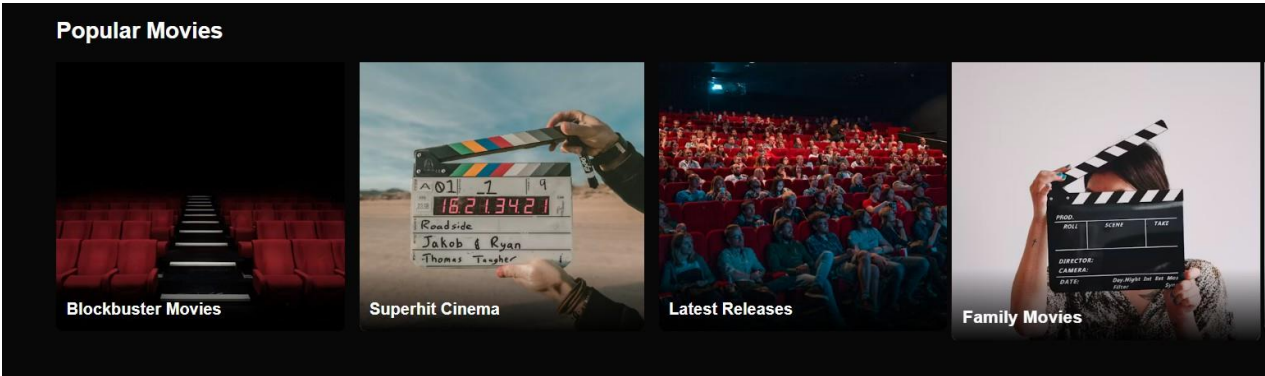


图 3 Popular Movies content row with category cards designed for visual browsing.

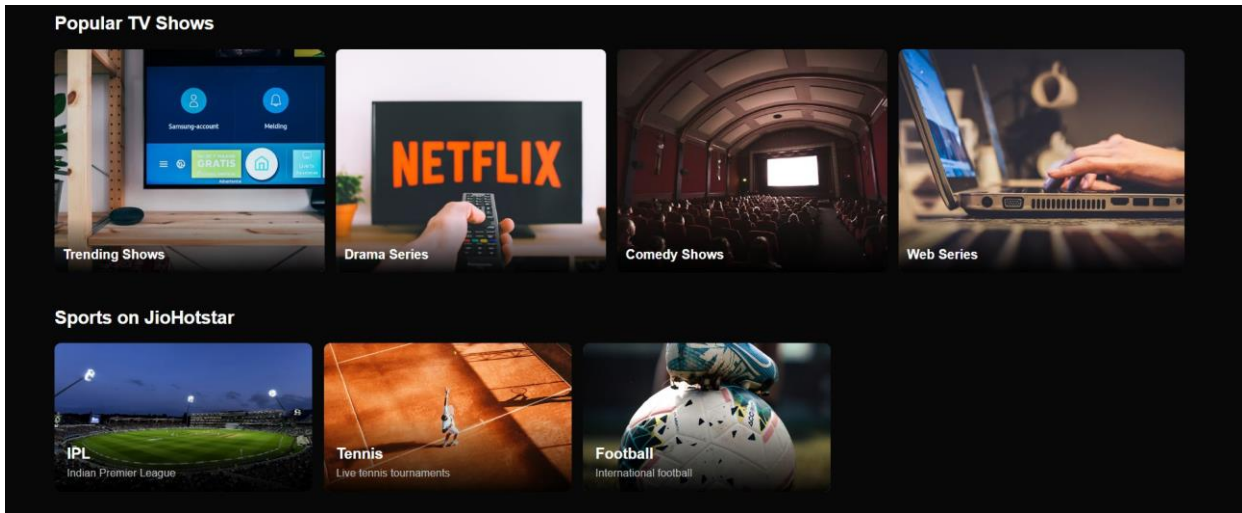


图 4 Popular TV Shows and Sports on JioHotstar sections showing reusable content-card layouts.

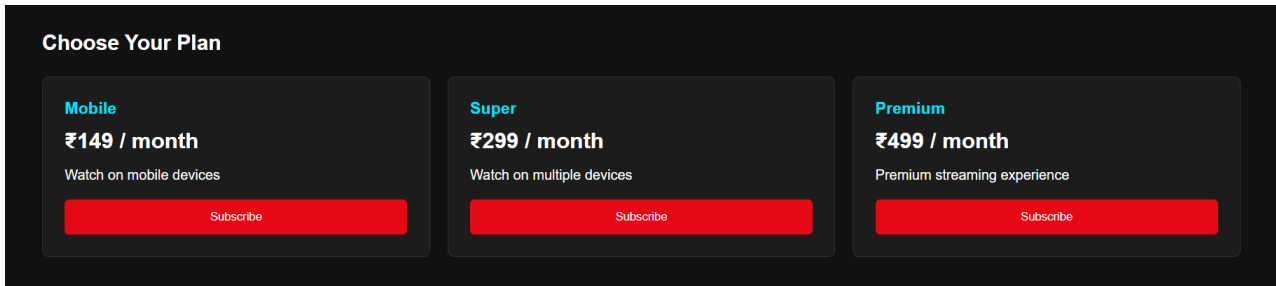


图 5 Subscription-plan section with Mobile, Super, and Premium plan cards.

The images are included as project evidence rather than as external dependencies. In a production build, these assets would normally be stored in version control or produced by the frontend build pipeline, then copied into the image as a reviewed release artifact.

4 Phase 1 — Server preparation

4.1 `sudo su`

`sudo` executes a command with elevated privileges according to the host's `sudo` policy. When followed by `su` without a username, the command normally opens a root shell. This makes subsequent commands run as root until the shell is exited.

The command is useful for short-lived administration on a disposable lab server, especially when several privileged commands must be executed. In production, a persistent root shell is usually discouraged because it removes repeated visual reminders that a command is privileged and increases the blast radius of a typo or pasted command. Prefer `sudo command` for each administrative operation, or use a tightly controlled administrative role. Do not disable auditing or share root credentials.

4.2 `yum update -y`

`yum` is a package-management interface used by several RPM-based distributions. `update` refreshes installed packages to available versions, while `-y` automatically answers yes to prompts. On newer distributions, `dnf` may be the preferred underlying tool, although `yum` can remain a compatibility command.

Updating reduces exposure to known defects and security vulnerabilities, but a production update is a change-management event rather than a command to run blindly. First test the update in a staging environment, review repository sources, take a recovery path or snapshot, apply maintenance windows, and reboot if the kernel or critical libraries require it. For reproducibility, record the operating-system image, repository configuration, update date, and resulting package versions.

4.3 `yum install docker -y`

This installs the Docker package and its dependencies from the configured repositories. The package name and repository availability vary by operating system and vendor. A production installation should follow the target distribution's official Docker Engine instructions, verify repository authenticity, and select a supported version instead of assuming that the default repository contains the desired release.

Installing Docker on a server is appropriate when the host is intentionally managed as a container host. It is not a substitute for host hardening. Restrict administrative access, apply security updates, configure disk and inode monitoring, and understand that membership in the `docker` group is effectively highly privileged because Docker can control the host.

4.4 `which docker`

`which` searches the shell's command path and prints the executable selected for `docker`. It is a quick diagnostic that confirms whether the command is discoverable and can reveal an unexpected installation path.

In production troubleshooting, use it to detect `PATH` problems, multiple installations, or a command resolving to an unexpected binary. It is not a security verification by itself; also inspect package ownership, version, service configuration, and file permissions when provenance matters.

4.5 `docker --version`

This prints the Docker CLI version. It confirms that the client is installed and provides an initial compatibility clue when diagnosing build or runtime behavior.

Production teams should record the Docker Engine and CLI versions in host inventory and upgrade them through a tested process. The CLI version alone does not prove which daemon is being contacted; for a fuller diagnostic, inspect `docker version`, which reports client and server details.

4.6 `systemctl status docker`

`systemctl` controls and queries services managed by `systemd`. The `status` subcommand shows whether the Docker service is active, recent log lines, the process identifier, and startup state. It is a read-only health check.

Use it during installation validation, incident response, and post-maintenance checks. For deeper diagnosis, inspect the service journal, daemon configuration, disk space, `cgroup` settings, and container events. A service being active does not mean that the application is healthy; it only indicates that the Docker daemon process is running.

4.7 `systemctl start docker`

This starts the Docker service immediately for the current boot. It does not necessarily configure the service to start after a future reboot.

It is suitable after installation or after a controlled service stop. In production, starting the daemon should be part of host provisioning or configuration management, not an undocumented manual step. Verify that expected containers are restarted according to policy and that the host's firewall and storage are ready before accepting traffic.

4.8 `systemctl enable docker`

This enables the Docker service in `systemd` so it is scheduled to start during future boots. `enable` changes startup configuration; it does not, by itself, start a currently stopped service.

The paired operational pattern is often `systemctl enable --now docker`, which enables and starts the service in one explicit action. Whether Docker should start automatically depends on the host's role, recovery objectives, and security policy. If automatic startup is selected, test boot recovery and ensure that application restart policies do not create an uncontrolled traffic surge.

5 Phase 2 — Project workspace

5.1 `mkdir jioHotstar`

`mkdir` creates a directory. Here it creates a dedicated workspace for the application source and Docker build context. Separating application files from unrelated server files improves organization and reduces accidental inclusion in an image build.

For production, prefer a predictable directory layout owned by a deployment user, such as `/opt/jiohotstar/releases/<version>` or a CI workspace. Use permissions that prevent unauthorized modification, and keep secrets outside the build context. A directory name's capitalization is significant on Linux filesystems, so standardize naming conventions.

5.2 `cd jioHotstar`

`cd` changes the current shell directory. It matters here because the subsequent `docker build -t jiohotstar .` uses `.` as its build context, meaning the current directory is sent to the Docker builder.

In production, verify the working directory in automation and use an explicit, clean checkout. The build context should contain only required files. A `.dockerignore` file should normally exclude Git history, editor files, local dependencies, test artifacts, credentials, certificates, and large unrelated assets.

6 Phase 3 — Image definition and build

6.1 `docker pull nginx`

`docker pull` downloads an image from a container registry. Pulling `nginx` without a tag typically resolves the registry's default tag, commonly `latest`. The command can warm the local cache before a build, although `FROM nginx:alpine` in the Dockerfile also causes the required base image to be pulled if absent. For production, pin a deliberate tag and preferably a digest, such as `nginx:1.27-alpine@sha256:<digest>`, after validating the exact version. Use a trusted registry, authenticate where required, scan the image, monitor upstream security advisories, and rebuild regularly. Avoid treating `latest` as a stable release identifier.

6.2 `vi index.html`

`vi` is a terminal text editor. The command creates or edits the static landing-page entry point. It is reasonable for a quick experiment directly on a server, but manual editing on a production host creates configuration drift and weakens reviewability.

A production workflow stores HTML, CSS, JavaScript, and assets in version control, reviews changes through pull requests, runs automated tests and linting, builds from a clean commit, and retains the commit or release identifier in deployment metadata. The server should receive an artifact, not an unreviewed interactive edit.

6.3 `touch Dockerfile`

`touch` creates an empty file if it does not exist, or updates its timestamps if it does. It is used here to create the Dockerfile before editing it.

In a repository, the Dockerfile should be created and versioned as source code. File timestamps should not be used as the release identifier; use a commit SHA, signed tag, or build number instead.

6.4 Dockerfile instruction: `FROM nginx:alpine`

`FROM` selects the base image. `nginx:alpine` combines NGINX with Alpine Linux, producing a smaller image than many general-purpose distributions. Smaller images can reduce transfer time and attack surface, but they do not automatically make an image secure or compatible.

Production guidance is to pin the base image, scan it, subscribe to security updates, and rebuild when the base changes. Validate NGINX modules, libc behavior, TLS requirements, observability agents, and debugging procedures against the selected base. Keep the base image minimal, but choose operational compatibility over size alone.

6.5 Dockerfile instruction: `COPY . /usr/share/nginx/html`

`COPY` transfers files from the build context into the image. NGINX serves files from `/usr/share/nginx/html` in the official image, so this instruction places the landing page at the web server's document root. The broad `.` source is convenient, but production builds should copy only the built artifact or explicitly selected files. Add a `.dockerignore`, avoid copying secrets, and use a multi-stage build when a frontend must first be compiled. Review file ownership and permissions, and ensure that runtime configuration is injected safely rather than baked into source-control artifacts.

6.6 Dockerfile instruction: `EXPOSE 80`

`EXPOSE 80` documents that the image intends to listen on TCP port 80. It is metadata for users and tooling; it does not publish the port to the host and does not configure a firewall.

Use it to communicate the container contract and improve discoverability. In production, enforce exposure through the runtime or orchestrator, bind only to required interfaces, place public traffic behind a load balancer or ingress, and terminate HTTPS at an appropriate edge. If the application listens on another port, keep the Dockerfile metadata and deployment configuration consistent.

6.7 Dockerfile instruction: `CMD ["nginx", "-g", "daemon off;"]`

`CMD` supplies the default command when a container starts. NGINX is told to run in the foreground with `daemon off;`, which is essential because the container runtime tracks the main process; if that process exits, the container stops.

The JSON or exec form avoids an extra shell layer and handles signals more predictably. Production deployments should pair a foreground process with graceful shutdown behavior, a health check, structured access/error logs, resource limits, and a restart policy or orchestrator-managed replacement. Do not use a container as a reason to run multiple unrelated daemons in one process space.

6.8 `docker build -t jiohotstar .`

`docker build` reads the Dockerfile and build context, executes its instructions, and creates an image. `-t jiohotstar` assigns a human-readable repository/tag reference; the final `.` selects the current directory as the context.

The command is useful for producing a deployable artifact from source. In production, build in CI from a clean, reviewed commit; tag with an immutable version such as a commit SHA; use BuildKit; generate a software bill of materials; scan for vulnerabilities; and push the image to a private registry. Do not rely on a mutable `latest` tag for rollback or auditability. A reproducible build also requires controlling base-image versions, dependency versions, build arguments, and generated assets.

7 Phase 4 — Container launch

7.1 `docker run -d --name jio-hotstar-server -p 80:80 jiohotstar`

`docker run` creates and starts a container from the `jiohotstar` image. The flags have separate responsibilities:

Option	Meaning	Production consideration
<code>-d</code>	Runs detached in the background	Use service supervision and logs; do not confuse “detached” with “highly available”
<code>--name</code>	Assigns a stable human-readable container name	Helpful for operators, but orchestrators generally identify workloads by service and deployment labels
<code>-p 80:80</code>	Maps host TCP port 80 to container TCP port 80	Publicly exposes the service; bind intentionally, add TLS/ingress, and coordinate with firewall rules
<code>jiohotstar</code>	Image used to create the container	Use an immutable registry reference or digest in production

For a learning server, this is a clear demonstration of container publication. For production, add an explicit restart policy such as `--restart unless-stopped` only after understanding failure behavior, set CPU and memory limits, configure log rotation, define a health check, and deploy through Compose, Swarm,

Kubernetes, or a managed container platform when service discovery, rolling updates, secrets, and replica management are required.

8 Production readiness checklist

Control	Recommended production practice	Priority
Identity and access	Use named non-root administrative users, least privilege, MFA where available, and audited sudo. Avoid routine root shells.	Critical
Supply chain	Pin the OS, NGINX base image, and dependencies; scan and sign images; record provenance and SBOMs.	Critical
Build context	Add <code>.dockerignore</code> ; build from version control in CI; never copy secrets into the image.	Critical
Availability	Use health checks, restart/replacement policy, capacity limits, and a tested rollback path.	High
Network security	Use a firewall, TLS, security headers, restricted admin ports, and an ingress or load balancer.	Critical
Observability	Centralize access/error logs, collect metrics, alert on failures, and retain deployment identifiers.	High
Operations	Define patch windows, image rebuild cadence, backup/recovery expectations, and incident ownership.	High

9 Safer reference implementation

The following example keeps the educational intent while improving repeatability. The exact image digest should be selected and verified by the project's release process rather than copied blindly.

```
FROM nginx:alpine

# Copy only the reviewed, built site artifact.
COPY ./dist/ /usr/share/nginx/html/

EXPOSE 80

CMD ["nginx", "-g", "daemon off;"]
```

```
# Build from a clean checkout and immutable release identifier.
docker build --pull -t registry.example.com/jiohotstar:${GIT_COMMIT_SHA} .

# Run with explicit operational controls; adapt to the platform.
docker run -d \
  --name jio-hotstar-server \
  --restart unless-stopped \
  --memory 256m \
  --cpus 1.0 \
  -p 80:80 \
  registry.example.com/jiohotstar:${GIT_COMMIT_SHA}
```

This remains only a baseline. Public production traffic should normally use HTTPS, a reverse proxy or load balancer, a private image registry, automated deployment, and a platform capable of replacing unhealthy instances without manual SSH sessions.

10 Command summary

Command or instruction	Immediate role in this project	Production-level interpretation
<code>sudo su</code>	Open a root shell	Prefer scoped sudo and audited access
<code>yum update -y</code>	Update host packages	Tested, scheduled patch management
<code>yum install docker -y</code>	Install Docker	Use supported, verified repositories
<code>which docker / docker --version</code>	Verify CLI availability	Record client/daemon versions
<code>systemctl status/start/enable docker</code>	Inspect, start, and enable the daemon	Automated host provisioning and boot recovery
<code>mkdir / cd</code>	Create and enter workspace	Controlled release directories and clean contexts
<code>docker pull nginx</code>	Download base image	Pin, scan, and verify provenance
<code>vi index.html / touch Dockerfile</code>	Create/edit source files	Version-controlled source and reviewed changes
<code>FROM / COPY / EXPOSE / CMD</code>	Define image and runtime contract	Minimal, pinned, observable, secure image
<code>docker build</code>	Create image	CI build, immutable tags, SBOM, scan, sign

<code>docker run</code>	Create and start container	Managed rollout, limits, health, TLS, and recovery
-------------------------	----------------------------	--

11 Conclusion

The original sequence is valuable because it makes the container lifecycle visible: prepare the host, create a workspace, define the image, build the artifact, and run the process. Its main limitation is that it is a manual, mutable, single-host workflow. Production engineering adds controls around identity, reproducibility, supply-chain trust, networking, observability, availability, and recovery.

The most important practical upgrade is to move from “SSH to a server and edit files” to “commit source, build an immutable image in CI, scan and publish it, deploy a versioned release, verify health, and retain a tested rollback.” That change preserves the simplicity of the project while making the operating model auditable and repeatable.

12 References

- [1] Docker, “Dockerfile reference,” [<https://docs.docker.com/reference/dockerfile/>].
- [2] Docker, “docker build,” [<https://docs.docker.com/reference/cli/docker/build/>].
- [3] Docker, “docker run,” [<https://docs.docker.com/reference/cli/docker/container/run/>].
- [4] Docker, “Docker image digest,” [<https://docs.docker.com/reference/cli/docker/image/pull/#pull-an-image-by-digest>].
- [5] Docker, “Docker security,” [<https://docs.docker.com/engine/security/>].
- [6] NGINX, “NGINX Docker image,” [https://hub.docker.com/_/nginx].
- [7] systemd, “systemctl — Control the systemd system and service manager,” [<https://www.freedesktop.org/software/systemd/man/latest/systemctl.html>].
- [8] Docker, “Build checks,” [<https://docs.docker.com/build/checks/>].

Scope note. This guide explains the commands and their operational meaning. It is not a distribution-specific hardening standard. Validate package names, repository instructions, firewall rules, image versions, and compliance requirements against the chosen cloud provider, Linux distribution, and organizational policy.